

OOP (Object Oriented Programming)

* Why OOP?

By using OOP we are able to solve real-life problems.

We all normal people think in a object oriented way, so that's why we use OOP.

① What is object?

Object is something which has ① State ② Behaviour ③ Identity ④ Responsibility.

①① What is state?

State means giving value to the attribute.

State = attribute + value.

②① What is Behaviour? (verbs)

response given by object to outside the world.
(when — then format).

③① What is Identity?

Identity means uniqueness associated with particular object.

To differentiate the object which has same state & behaviour.

④① What is responsibility? (Adjective)

Role of a object in real-world.

1) def (object) —

e.g. Fan

②. what is state?

A) Object :-

Object is something which has

- ① State
- ② Behaviour
- ③ Identity
- ④ Responsibility.

e.g. Fan

① State :- State means giving value to the attribute
State = attribute + value.

e.g. ^{white} color, price = 500 ₹, ~~to~~ Brand = "LG"

② Behaviour :- Response given by object to outside the world.

e.g. Switch the Fan ON / OFF → giving air, cool air

③ Identity :- Identity means uniqueness associated with particular object.

e.g. ~~to~~ company (brand) . unique number.

④ Responsibility :- Role of a object - in real-world.

e.g. Rotating (give air)

B) object :-

Object is something which has

- ① State
- ② Behaviour
- ③ Identity
- ④ Responsibility.

e.g. car.

① State :- State means giving value to the attribute
State = attribute + value.
e.g. ^{black} color, price = 2,00,000, Brand: swift

② Behaviour :- Response given by object to outside the world.
e.g. speed, when we put key in the key holder than car gets started.

③ Identity :- Identity means uniqueness associated with particular object.
e.g. Tata, swift model, IMEI number

④ Responsibility :- Role of a object to real-world
e.g. safe travelling / riding.

c) object :- object is something which has

① State

② Behaviour

③ Identity

④ Responsibility.

e.g. Pen

① State :- state means giving value to the attribute
State = attribute + value.
e.g. price = 5₹, black pen, Brand = cello

② Behaviour :- Response given by object to outside the world.
e.g. ^{show} Fe written text.
when text written → it shows the text.

③ Identity :- Identity means uniqueness associated with particular object.

e.g. NATARAJ unique memory address

④ Responsibility :- Role of a object in real world
e.g. writing text or drawing.

Object :- object means something which has

① State

② Behaviour

③ Identity

④ Responsibility.

e.g. Notebook.

① State :- State means giving value to the attribute
State = attribute + value

e.g. price = ₹80, totalPages = 100, coverColor = Blue.

② Behaviour :- Response given by object to outside the real world.

e.g. good pages, new, clean

③ Identity :- Identity means uniqueness associated with particular object

e.g. ~~quick work~~ (Brand) Barcode / Serial number

④ Responsibility :- Role of a object in real world.
e.g. store data.

EJ object : object means something which has

- ① state
- ② Behaviour
- ③ Identity
- ④ Responsibility

e.g. shoes

① state : State means giving value to attribute.

state = attribute + value

e.g. price = 2000 ₹, color = black, size = 8 UK, Brand = Nike

② Behaviour : Response given by object to outside the world.

e.g. ~~comfort~~ clean, Laces

③ Identity : Identity means uniqueness associated with particular object,

e.g. Bata, Latur shoes memory reference.

④ Responsibility : Role of a object in real world

e.g. protect ↓ from dust
the foot

- There are 4 steps of oop language :-

- 1) Abstraction
- 2) Encapsulation
- 3) Inheritance
- 4) Polymorphism

Q what is abstraction?

→ Abstraction is the process of selective Ignorance.

- Abstraction is the process of requirement Gathering which is present in SDLC (Software Development Life Cycle).

- It is a completely pen & paper process.

- we can not write a single line of code for an abstraction.

Note : abstraction & abstract class is a different class

example :- 1) Person (object)

Employee	Person	Patient
Name, address,	Name, address, age,	Name, address,
age, gender,	gender, education,	age, gender,
education,	weight, height, skills,	weight, height,
skills, Hobbies,	BG, color, Hobbies,	BG, color,
nationality,	nationality, caste,	caste
Caste, disability,	disability, marital - status,	marital - status,
marital - status,	mother tongue, Asset,	disability, M.No.
mother - tongue,	resignation, mo.No.	
resignation,		
M.No.		

→ Defⁿ :- Abstraction is the process of selective ignorance.

- Abstraction is the process of requirement Gathering which is present in SDLC.
- It is Completely pen & paper process
- we can not write a single line of code for abstraction.

Example 1: Vehicle (object)

RTO /	Vehicle	Driver
Registration office	Model, VIN	Current Speed
Model,	License Plate,	Fuel level,
Make,	Color, Current	Engine
VIN,	Speed, Fuel	temperature,
License	level, engine	odometer
Plate,	Temperature,	Reading.
Color,	Cargo	
Insurance	Capacity.	
Policy No.	Insurance	
	Policy No	
	Maintenance	
	History.	
	odometer	
	Reading	

2) Defⁿ :- Abstraction is the process of selective ignorance.

- Abstraction is the process of requirement Gathering, which is present in SDLC.
- It is Completely pen & paper process.
- we can not write a single line of code for abstraction.

Example 2 : Bank Account (object)

Bank Account

ATM User

Bank Manager

ATM User	Bank Account	Bank Manager
Account Number, Balance, Security PIN, Transaction History.	Account Number, Holder Name, Balance, Security PIN, Nominee, Transaction History, Government ID (PAN) Interest Rate, Branch Code, Overdraft Limit, Linked debit card Number.	Account number, Holder Name, Balance, Nominee, Government ID, Interest Rate, Overdraft Limit.

3) Definition (Abstraction) :-

- Abstraction is the process of selective ignorance.
- Abstraction is the process of requirement gathering which ~~has~~ is present in SDLC.
- It is completely pen & paper process.
- We can not write a single line of code for abstraction.

Example 3 : Book

Reader / Buyer		Book	Store Inventory Manager
Title, Author, Retail Price, page Count, Genre, Summary, Customer Reviews.		Title, Author, ISBN, Retail Price, Cost Price, Stock Quantity, Publisher, Page Count, Genre, Summary, Warehouse, Shelf location, Customer Reviews, Publication Year.	Title, ISBN, Retail Price, Cost Price, Stock Quantity, Publisher, Warehouse, Shelf location.

1) Definition (Abstraction) :-

- Abstraction is the process of selective ignorance.
- Abstraction is the process of requirement gathering which is present in SDLC.
- It is completely pen & paper process.
- we can not write a single line of code for abstraction.

Example 4 : Smartphone

Smartphone

End-User

Service

Center

Technician

Battery
Percentage
Wi-Fi
Signal
Current
Active
Background
Apps

Brand, Model,
Battery, Percentage,
wifi signal
IMEI, Number,
Total storage
Capacity, Screen,
Resolution,
Camera
megapixels,
Current
Active Background
Apps, Serial
Number, OS
version.

Brand, Model,
IMEI No.
Total
storage
Capacity,
Serial
number,
OS
version

5> Definition (Abstraction) :-

- Abstraction is the process of selective ignorance
- Abstraction is the process of requirement gathering which is present in SDLC.
- It is completely pen & paper process
- We can not write a single line of code for abstraction.

Example 5 : Product

product

Online
Shopper

Name,
Selling
price,
Expiry
date,
Active
Discount
Code.

product ID,
Name, Cost
Price, selling
price, Supplier
Contact,
weight,
Package, Dimen-
sions, Expiry
date, Active
Discount Code,
Barcode
Number,
Warehouse
Aisle
Section

Supply chain

product
ID, weight,
Package
Dimensions,
Barcode
Number,
Warehouse
Aisle
Section,
Supplier
Contact

Encapsulation Examples

① class Emp:

```
def __init__(self, id, name, sal):
```

```
    self.id = id
```

```
    self.name = name
```

```
    self.sal = sal
```

```
def display(self):
```

```
    print(f'Added Employee : \n id = {self.id} \n
```

```
    name = {self.name} \n sal = {self.sal} \n')
```

```
e1 = Emp(21, "Sachin", 210000)
```

```
e1.display()
```

② class vehicle:

```
def __init__(self, model, name, price, capacity):
```

```
    self.model = model
```

```
    self.name = name
```

```
    self.price = price
```

```
    self.capacity = capacity
```

```
def display(self):
```

```
    print('***** Vehicle Details*****')
```

```
    print(f'Model = {self.model} \n Car name =
```

```
    {self.name} \n Price = {self.price} \n
```

```
    capacity = {self.capacity}
```

```
v1 = vehicle('creative(2026)', 'Tata Nexon', 800000,
```

```
    '5 Seater')
```

```
v1.display()
```

O/p: ***** Vehicle Details*****

Model = creative(2026)

Car name = Tata Nexon

Price = 800000

capacity = 5 seater.


```

③ class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price
    def display(self):
        print('***** Book Details *****')
        print(f'Title = {self.title} | Author = {self.author} | Price = {self.price}')
b1 = Book('Library Book Selection', 'S.R. Ranganathan', 4000)
b1.display()

```

```

④ class Mobile:
    def __init__(self, brand, model, price, os):
        self.brand = brand
        self.model = model
        self.price = price
        self.os = os
    def display(self):
        print('***** Mobile Details *****')
        print(f'Brand = {self.brand} | Model = {self.model} | Price = {self.price} | OS version = {self.os}')
m1 = Mobile('OnePlus', 'OnePlus Nord CE 6 5G', 59000, 'Android 16')
m1.display()

```


⑤ class Product :

def __init__(self, id, name, price):

self.id = id

self.name = name

self.price = price

def display(self):

print('***** Product Details *****')

print(f'Product ID = {self.id} In Product

name = {self.name} In Price = {self.price}')

p1 = Product(101, 'Snacks', 500)

p1.display()

⑥ class BankAccount :

def __init__(self, AcNo, HolderName, Pin):

self.AcNo = AcNo

self.HolderName = HolderName

self.pin = pin

def display(self):

print(f'***** Bank Account Details *****')

print(f'Account Number = {self.AcNo} In

HolderName = {self.HolderName} In Pin =

{self.Pin}')

b1 = BankAccount(43512057177, 'Neha pawar', 7759)

b1.display()

o/p: ***** Bank Account Details *****

Account Number = 43512057177

Holder name = Neha pawar

Pin = 7759

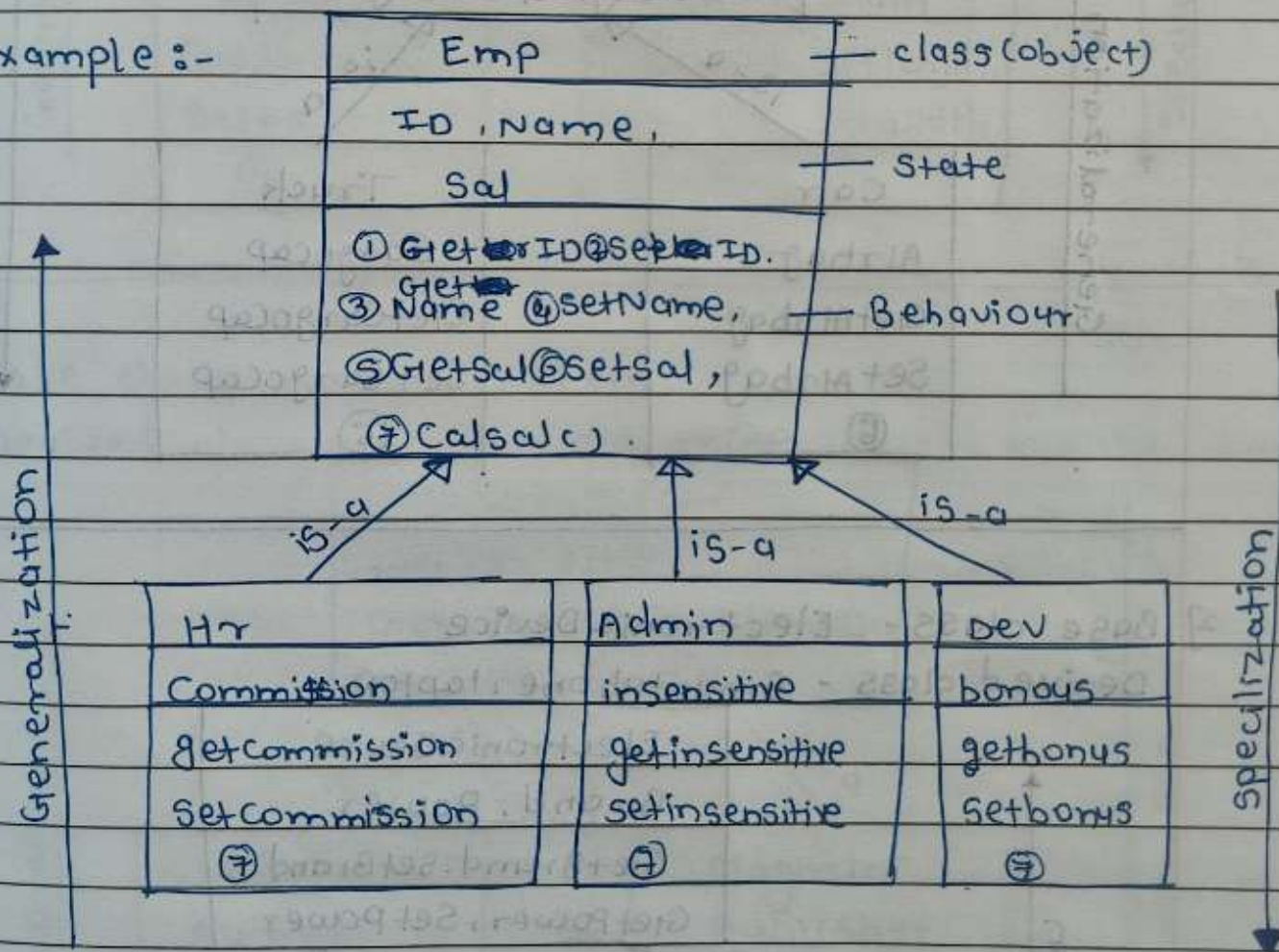
3] Inheritance :-

Reuse of already define classes with "is-a" relationship.

Q. Why their is need to reuse ?

→ To reduce the code redundancy. If their is a code redundancy in your program so their is high chances to System Failure.

• Example :-

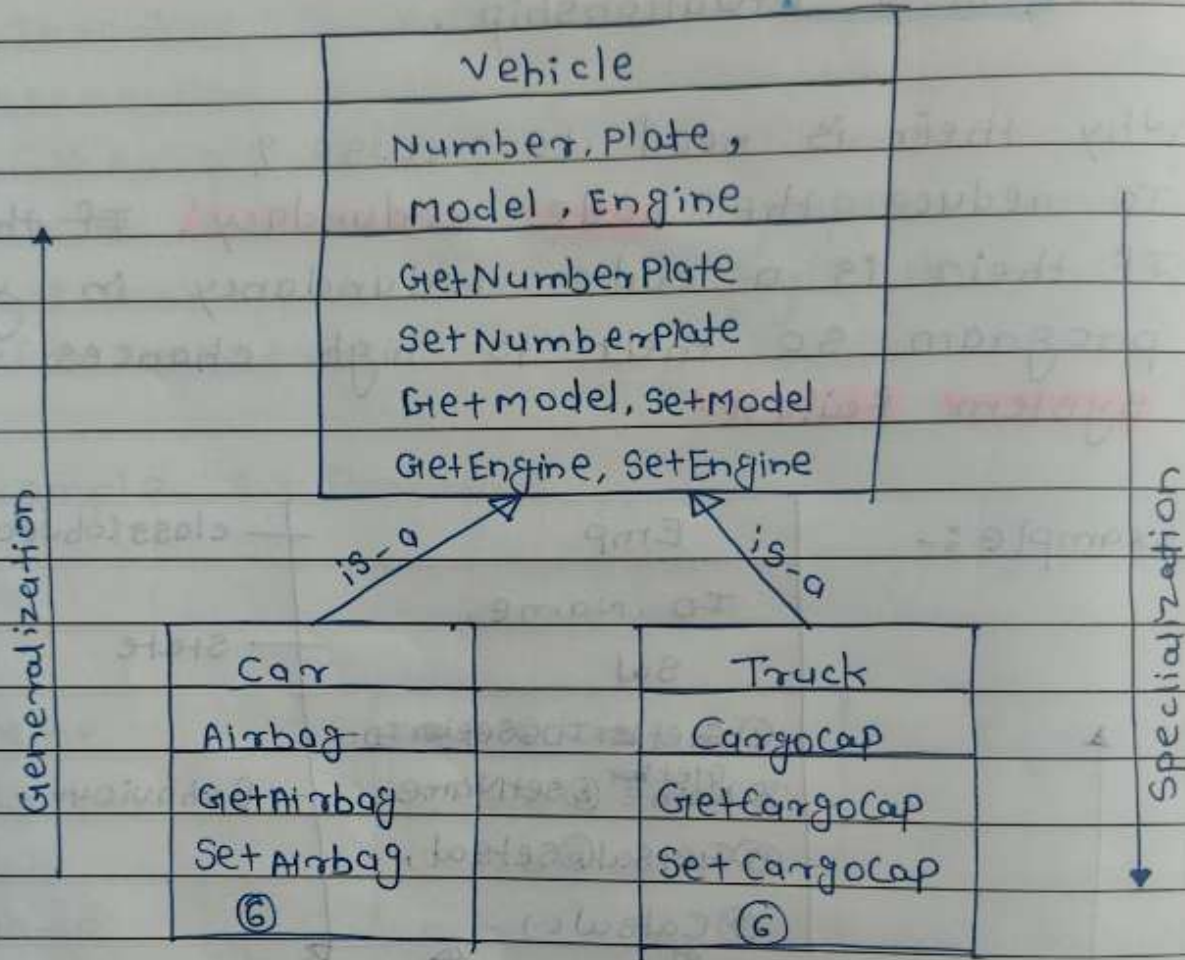


Upward to the herachy ^{we get} generalization & Down ward to the herachy ^{we get} specilization.

Inheritance Examples

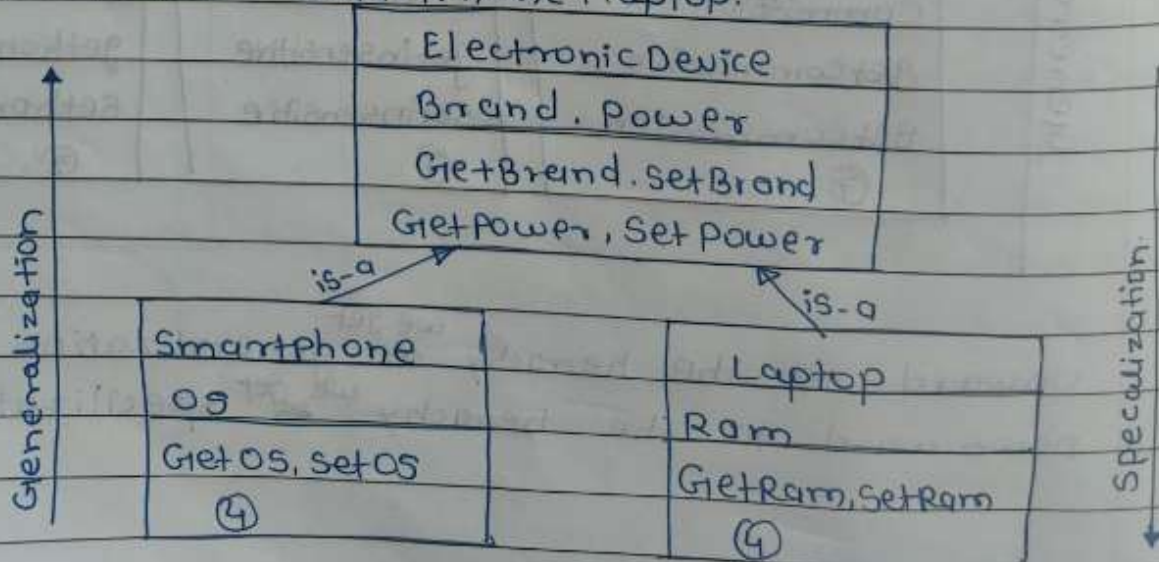
1] Base class - vehicle

Derived class - car, Truck,



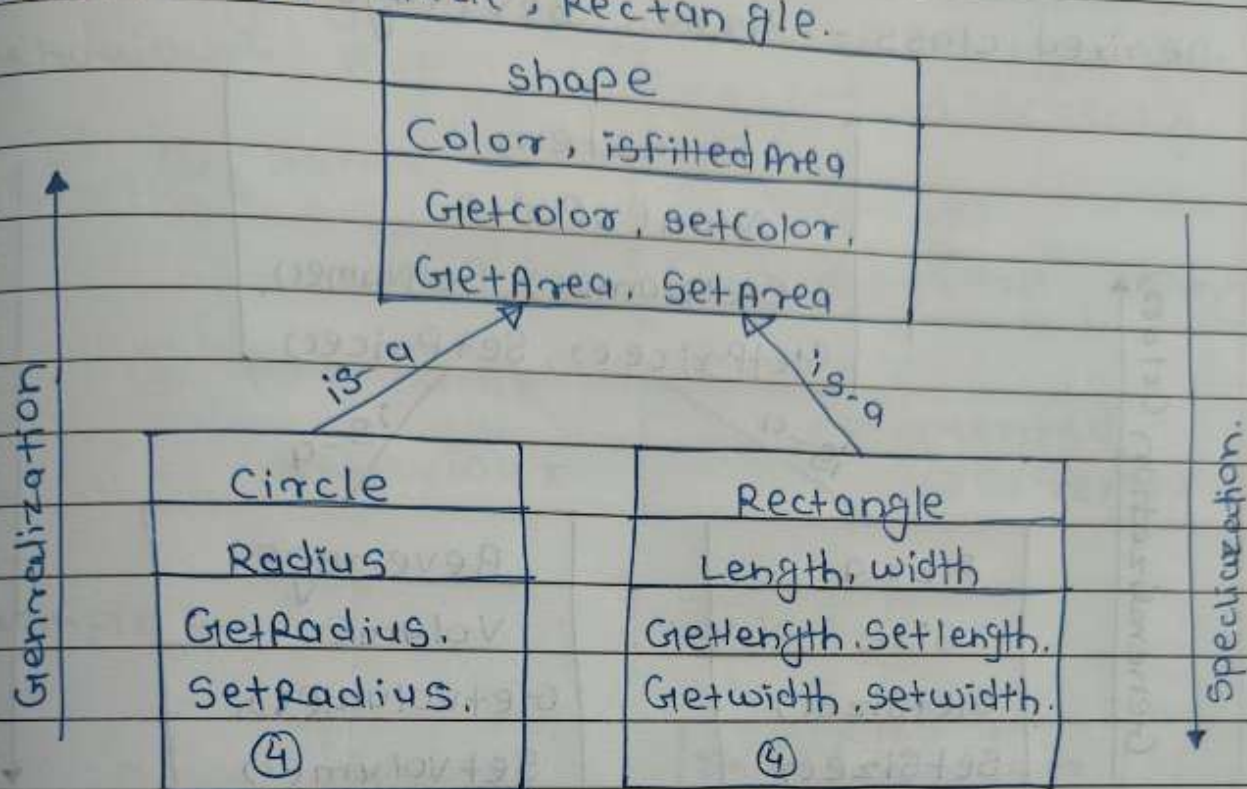
2] Base class - Electronic Device

Derived class - Smartphone, Laptop.



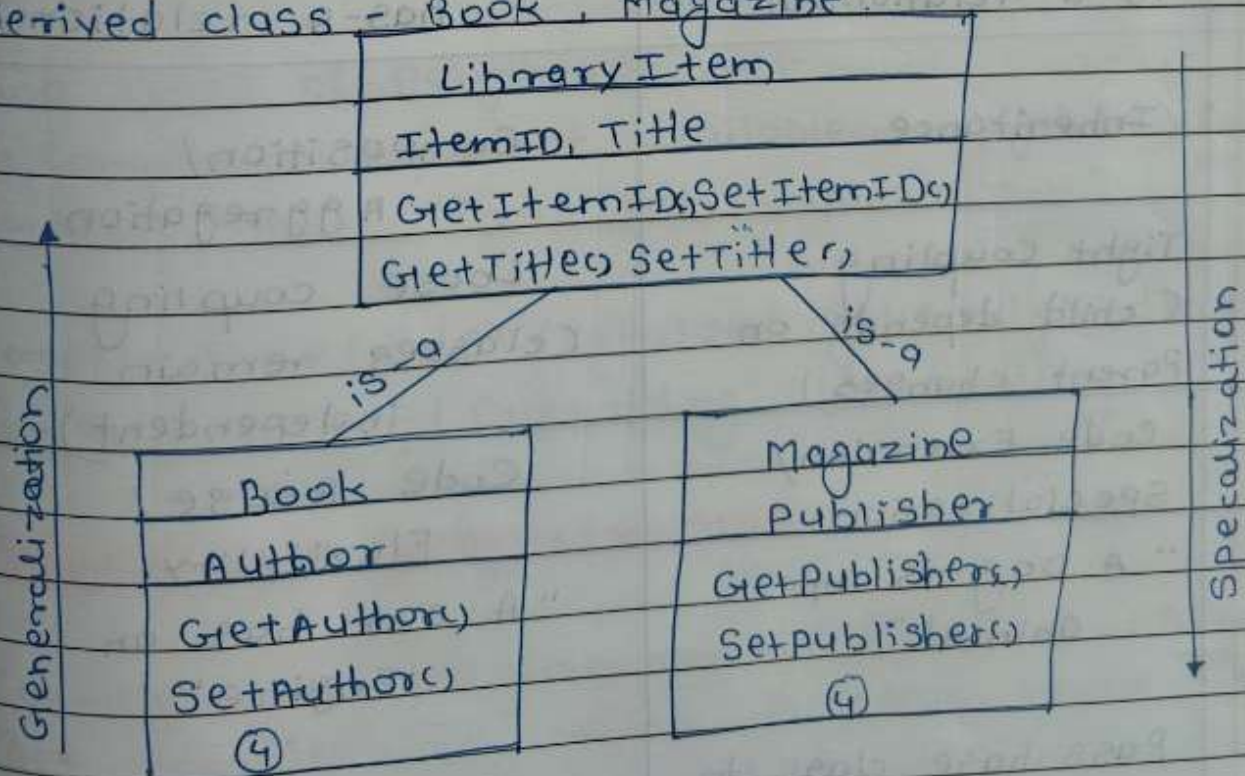
3] Base class - Shape

Derived class - circle, Rectangle.

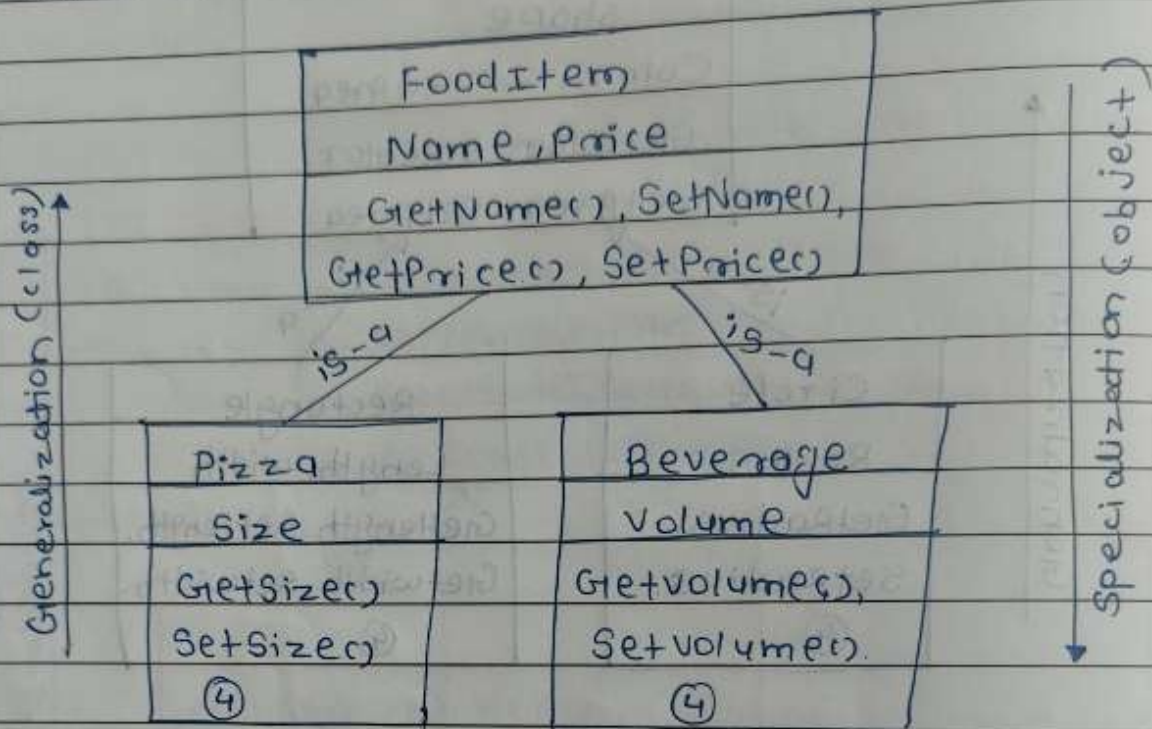


4] Base class - LibraryItem

Derived class - Book, Magazine



5) Base class - FoodItem
 Derived class - Pizza, Beverage



Feature	is-a relationship	has-a relationship
oop	Inheritance	Composition/
Concept		Aggregation
Coupling	Tight Coupling (child depends on Parent changes)	Loose coupling (classes remain independent)
design	Code Extension/ Specialization	Code Reuse/ Flexibility
Real-World	"A Dog is an animal"	"A car has an Engine"
Analogy		
python	Pass base class to derived class definition	Instantiate class inside another class
Syntax		

4] Polymorphism :-

Page No.:

Date: 20-09-26

M T W T F S S

Q. What is polymorphism?

Same msg given to generalize things for same behaviour But implemented differently.

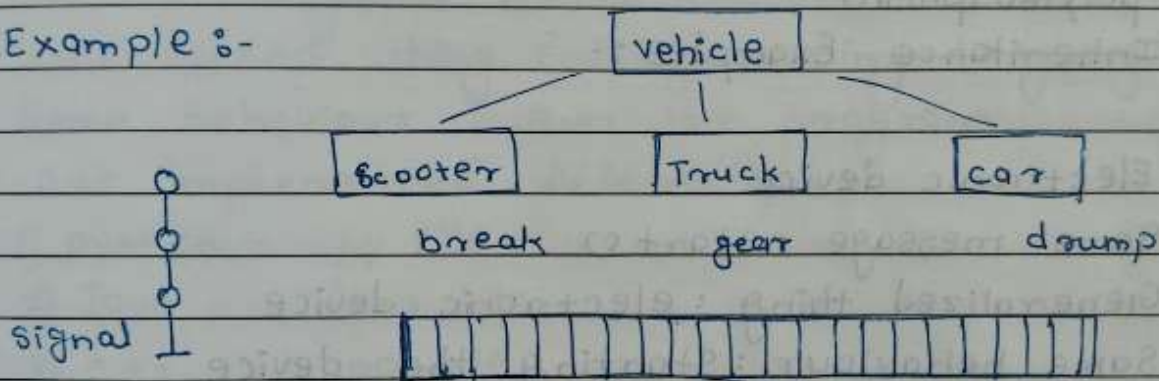
① Same msg

② given to generalize things

③ for same behaviour

④ But implemented differently.

Example :-



• two types of polymorphism :-

- 1) Compile-time / (Not available in python) false Polymorphism
- 2) Run-time / True polymorphism

• Compile-time / false / overloading / early binding

• Run-time / True / Overriding / late binding

1] Compile-time polymorphism :-

- In Compile-time polymorphism & functions, signature get consider in which function name must be same & number of parameter must be different as follows :-

- 1) sequence of parameter must be different
- 2) sequence of datatype of parameter must be different.
- 3) No. of parameters must be different.
- 4) datatype of parameter must be different.

- Compile-time polymorphism is done by the process of name manually mangling.

polymorphism

* Inheritance Examples :-

1] Electronic device

Same message : start()

Generalized thing : electronic device

Same behaviour : Starting the device

But implemented differently:

1] Mobile phone - power button.

2] AC - remote

3] T.V. - switch on

2] class - music

Same message : sing()

Generalized thing : music

Same behaviour : singing song / music

But implemented differently:

1] classical music

2] Folk music

3] Rock music

3] class - event

Same message : decorate()

Generalized thing : event

Same behaviour : decorate the hall according to event

But implemented differently : 1] wedding event

2] birthday event. 3] Engagement event.

4] class - Programming language

Same message : Run()

Generalized thing : Programming language

Same behaviour : Run the program

But implemented differently :

1] python - .py File

2] Java - .Java File

3] c++ - .cpp File

5] class - Bank Account

Same message : open()

Generalized thing : Bank Account

Same behaviour : open bank account

But implemented differently :

1] Saving Account -

2] checking Account current Account

3] Fixed deposit Salary account.

6] class - FoodItem

Same message : Expiry date()

Generalized thing : FoodItem

Same behaviour : Expiry date of FoodItem

But implemented differently :

1] Fresh produce 2] Packaged snack 3] Frozen Food